Unauthorised JavaScript Injection on Public Websites response

Web / API · **Critical** severity · SOC IR Playbook

Incident type	Web Application Compromise - Script Injection
Severity	Critical
Priority	Critical
Detection sources	WAF, SIEM, Website Monitoring Tools, Bug Bounty Reports, Client Feedback, Threat Intel Feeds

Scenario

An attacker injects malicious JavaScript code into a public-facing website (via compromised CMS, third-party scripts, misconfigured CDN or direct file replacement). This could lead to credential harvesting, skimming (e.g., Magecart), clickjacking, redirection to malicious sites or session hijacking.

MITRE ATT&CK

T1059.007, T1185, T1189, T1557.002

Preparation

Implement CSP (Content Security Policy) Restrict unauthorised scripts from loading Monitor file integrity Use tools to track changes in JS files on production Use Subresource Integrity (SRI) For third-party scripts to ensure they aren't tampered Enable Web Application Firewall (WAF) Block suspicious inputs or exploit attempts Perform regular code audits Especially on CMS plugins and third-party inclusions

Detection & Analysis

Alert from WAF or monitoring tool Detection of injected or modified JavaScript Validate file changes Compare modified JavaScript to known good versions (git, backups) Review injection source Was it from a CMS plugin, third-party source or direct code edit? Examine script behaviour Analyse the payload: keylogging, exfiltration, redirection, data capture

Containment

Remove or replace injected script Immediately restore clean versions from backup or repository Block malicious domain If external scripts were involved, block via DNS, proxy or firewall Disable affected parts of the site Temporarily take down the compromised section or page if necessary Alert customers/users If data harvesting occurred, communicate the exposure risk quickly

Eradication

Identify root cause Compromised admin credentials? Insecure plugin? Third-party breach? Patch CMS or plugin Apply updates and disable unnecessary or untrusted components Replace compromised components Reinstall from official sources with verified integrity Clean residual access Change admin credentials, revoke tokens, check server logs for persistence techniques

Recovery

Validate website integrity Recheck all files and scripts for correctness and cleanliness Resume normal operation After confirming no malicious code remains Perform vulnerability assessment Especially on exposed CMS, JavaScript includes and APIs Monitor for repeat attempts Increase web traffic and behaviour monitoring temporarily

Lessons Learned & Reporting

Document the incident Include injection vector, impact, attacker domain and mitigation steps Update web app monitoring rules Add new indicators of compromise for JavaScript integrity alerts Train web developers and admins On safe script practices, plugin security and change control Report to regulators If user data or payment information was compromised Improve SDLC security Integrate code scanning, dependency checks and CI/CD validation in development workflows

Tools typically involved

Web Monitoring Tools (e.g., Detectify, JSWatcher, SilentPush, Snyk, Sucuri); SIEM (e.g., Splunk, Sentinel, QRadar); Web Application Firewall (e.g., Cloudflare WAF, AWS WAF, Imperva); CMS platforms and source repositories (e.g., WordPress, GitHub); File integrity monitoring (e.g., OSSEC, Tripwire)

Success metrics (SLA targets)

Detection Time <15 minutes from script modification or alert Script Removal Time <30 minutes after detection Website Restoration Time <2 hours if critical path is affected Impact Notification Time Within 24 hours (or regulatory SLA) Repeat Attack Monitoring Duration Minimum 7 days of enhanced surveillance